Depth-Sensing Boat

Flaming Rubber Duckies
Connor Ford, Medhaj Shrestha, Ryan Slocum, Jacob Spagnolli

GEEN 1400-50 – Knutsen

I. Abstract

We are the Flaming Rubber Duckies and our project is the depth-sensing boat (DSB). The boat is a small watercraft made of acrylic and rubber. It uses Bluetooth technology to control the boat's power and steering, as well as its winch system. We used a brushless motor with a small propeller to generate thrust, similar to a fan boat, and servos for the winch and rudder systems. The boat uses the winch to drop a small compartment of sensors to the bottom of the body of water, gathering data on the depth, pressure, temperature, and GPS location. This data then gets written to an SD card, and this data is later parsed and analyzed in Matlab to produce a rough 3D map of the body of water, with shading along the z-axis for the depth and GPS coordinates for the x and y coordinates.

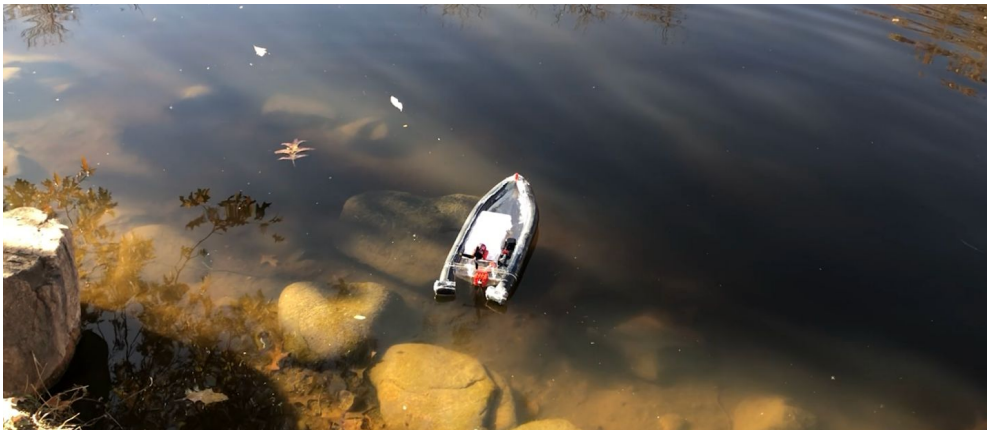
The final design of the DSB worked well with regards to the functionality of the boat and winch system, but the method of retrieving the pressure and depth data was inefficient and risky. By requiring the need to lower a container down in the water, the risk of damaging the sensors and other electronics was greatly increased. The container hosting the electronics to be lowered had to be completely waterproof, and at certain times this proved to not be the case as we encountered leaks in the container. The container also had to be heavy enough to sink and this caused a problem of not being able to comfortably fit all the electronics into the container because a filler was used to increase weight and reduce air. This heavy container also made a center of mass that was more towards the front, putting the boat in jeopardy of taking on too much water and sinking while accelerating forward and bringing back up the container from the water. We recommend using a different way to retrieve depth data rather than dropping a pressure sensor into the water. Overall the final design of the boat worked, hitting on our goal of creating a boat that reads and analyzes the depth, pressure, and temperature data of a body of water.

II. Introduction

When our group came together to come up with a project idea we were struggling to find something to put our effort into. We decided that some underwater mapping project would be interesting, but quickly learned that sonar-type technologies cost hundreds of dollars and were out of reach for our small introductory engineering project. Upon doing some more in-depth research we stumbled upon a depth sensor that was a lot less expensive, but it was small and would have to be lowered into the water to take readings. This seemed like the perfect challenge for our group to take on, and it satisfied the requirements outlined, so we got right to work. Our project's end goal became making a small remote controlled boat that would be able to lower the depth sensor into the body of water, take readings of the pressure, depth, temperature, and GPS locations at those points, and use all of that data to map the bottom of small bodies of water like ponds, lakes, or other small and slow moving bodies of water.

Our project would also prove to be very useful in the engineering, building, and data collection worlds. As stated, sonar-type depth sensors are very expensive and can be hard to use in smaller bodies of water where using a larger boat is not an option. Being able to map the depth of bodies of water remotely via our small craft offers a great solution to this issue, reducing both labor and cost. The small remote control boat we made is significantly cheaper than any sonar sensor out on the market today, and it removes the need for a larger aquatic craft to take readings. All the data being collected by the boat is also easy to interpret after running the MATLAB code we produced to visualize the body of water. There are many uses for the information pulled by this boat, one of which is research. There are many examples of research and studies being conducted that would benefit from a cheap and easy way to visualize the bottom of a body of water. Whether the testing is about the geographical nature of the aquatic environment or the creatures living inside of it, our boat would be hands down the best way to receive the data. The boat could also prove helpful in architectural and engineering pursuits, allowing engineers and contractors to measure the depths of the bodies of water they are building around. Unsolid ground and water tables can prove to be disastrous for civil and architectural engineers if unknown. Our boat provides a simple and quick solution to this issue, and could potentially help architects and engineers around the world.

Our project, while initially intended to just function as a cheaper alternative to sonar-type depth sensors, ended up being a completely separate and useful device in its own capacity. The boat functions as a perfect intermediate device for depth measurement of bodies of water when it is too small for larger sonar craft, but too big to visually inspect it. We hope that our project, when fully operational and optimized, could help engineers, builders, architects, and environmental researchers in their daily lives.



III. Design Requirements

The prompt for our project was to design a system that used an Arduino and sensors to respond to its environment. This gave us a lot of freedom, so we decided to build a small boat capable of mapping the bottom of a body of water.

The first requirement we had for the project was to build a boat that was bluetooth controlled, and could navigate a small body of water using a propulsion system and a steering system. This base for our project would allow us to swiftly move across the lake or pond, and collect data points for our map, all remotely.

The second requirement for our project was that it would sense depth, latitude, and longitude, and record them in a way that we could use for later analysis. The first idea for this would be a livestream of data back to the laptop, but this proved too difficult, so we ended up needing a SD card writer to record data for later use.

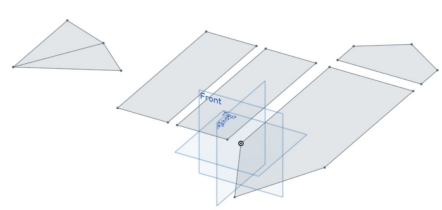
Finally, we needed software that would be capable of taking the data we recorded, and putting it into a three dimensional map that would model the contours and depth of the lake or pond we were exploring. We ultimately decided that MATLAB would be the ideal tool for this.

IV. Design Alternatives and Process

When we began this project, we started out with a clear set of steps and priorities to take:

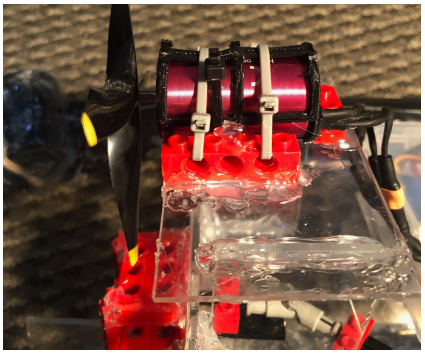
1. Build a small boat that floats on the water
2. Add a control system to the boat that consists of propulsion and steering
3. Implement bluetooth control to drive the boat from the shore using a laptop
4. Create a sensing assembly capable of taking data on depth and location
5. Put the sensing assembly and the boat together using a winch system
6. Develop mapping software to take data and produce a visual representation

These steps gave us a clear roadmap of what we needed to do. We started by building a basic prototype of our boat out of cardboard to test the specifications and design of the hull of the boat. After we decided that our hull design was exactly what we were looking for, we built the full scale model out of 1/8in thick acrylic, a silicon sealant, and a bike tube. This design was inspired by a U-shaped boat or dinghy, which was chosen based on its simplistic design and stable build. We glued all the pieces together using hot glue and sealed them with the silicone adhesive. Through the entire process of working on the boat the hull leaked at several different points, requiring it to be patched with more silicone. After the third time of these leaks happening we decided to ditch the silicone on that portion of the hull and replace it with JB Weld marine adhesive, which ended up working much, much better. In hindsight we should have just used the marine weld in the

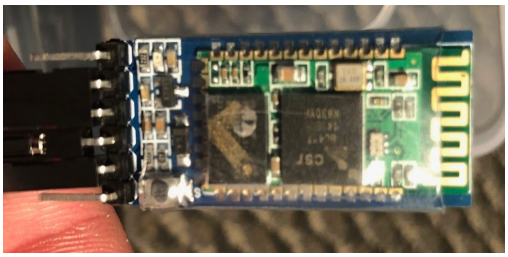


first place, it would have saved us a lot of time and a little bit of money. The bike tube part of the hull worked very well, we just had to pump it back up every few days, so if we were to redo this section of the project we probably would have found a better way to seal the cut ends of the bike tube so we wouldn't have had any issues with air leaks.

Moving on to the second phase of the project, we needed to add a propulsion and steering system to the boat that would end up being controlled via bluetooth. To start with the propulsion system, Dr. Knutsen generously allowed us to use his brushless motors to create a fan motor for the boat, Our original design was to have the fan point down a piece of PVC piping and into the water to push the boat through the water. After trying out this design we realized that we had made two critical design errors. The first was that there was no easy way to connect the PVC to the boat, and the second being that there was no fan powerful enough to propel the boat that would fit inside the PVC. In addition to that we were having trouble coming up with a good way to mount the brushless motor so that there would be enough air flow for the fan. We scrapped this design all together and pursued our second design, which you can see in the photo above. We started out attempting to use a 3D printed propeller, but when we realized it was not symmetrical and highly unstable, we decided to just purchase a propeller online and use it insead. The brushless motor was mounted on an acrylic platform elevated to make sure the propeller didn't hit anything, and the propulsion side of the boat's controls was complete. Moving on to the steering system we decided to go with one simple rudder centered on the back of the boat. Originally our plan was to mount a servo right on the back of the boat under the propeller, but quickly we decided this was a bad idea due to how easy it would be to get the servo wet. We then decided to build a small drive train on the boat that allowed the servo to sit atop the boat and towards the middle instead of way in the back. We used lego technic type pieces to make the axel, and built a joint using lego gears to convert the axel turning direction from out the back of the boat to down into the water. We then glued an acrylic rudder that we laser cut to the part of the axel and the drivetrain was completed. The wiring for the brushless motor and rudder servo both connected to the bluetooth sensor mentioned in the next paragraph to control the components from a laptop. Our final design for the propulsion and rudder system was probably the best functioning part of the entire project.



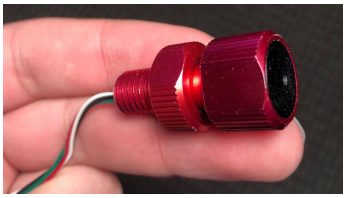
Next, we had to use the bluetooth modules we bought to wirelessly control the boat. We initially wanted to do this by building a small remote control out of an arduino and a bluetooth module, but after a lot of testing with the module, we decided that was not feasible. Instead, we opted to control the boat using a laptop. We spent several days trying to figure out how best to send and receive data using the bluetooth modules we bought, to no avail. Finally, we stumbled upon a resource online that suggested the Processing IDE, a Java based IDE that made graphical user interfaces and serial communication easy to work with. After trying this interface, we were finally able to use the bluetooth modules and arduino to



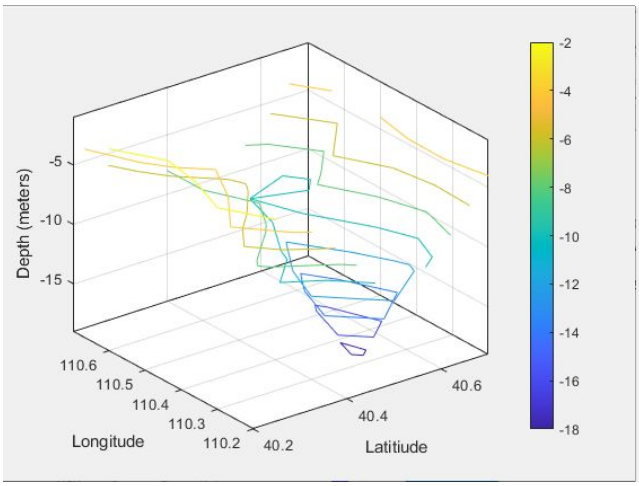
control an LED remotely. We quickly figured out how to implement keyboard controls, and we eventually got a system that would send data to the arduino about how to operate the motor, rudder, and winch. However, we had a major challenge: we only had one byte of data to work with. This meant that we could send the numbers 0 to 255 to the arduino, but that was it. So we had to use some code that would take keyboard inputs, encode them into a number between 0 and 255, send that number to the arduino, and decode them into motor outputs. This was a very convoluted system, and we were lucky that we only had three motors to control, otherwise we would have needed a different communication system. We would not recommend using these bluetooth modules (HC-05).

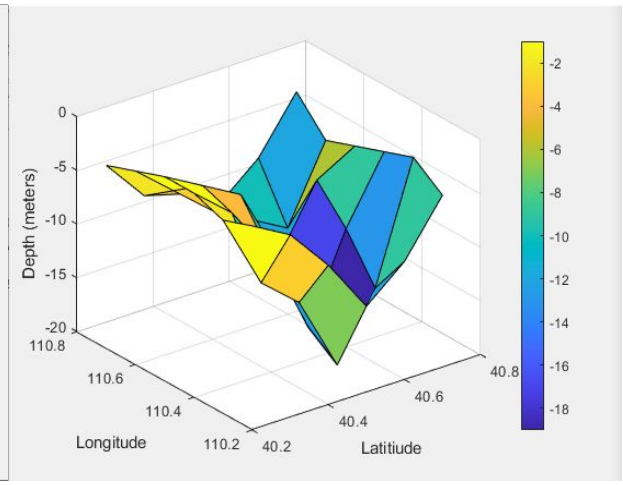
Our biggest challenge was certainly the sensing assembly. By the end of the project, we had all of the components inside of it working correctly (Arduino Nano, GPS module, depth-sensor, SD card writer), but there were two main issues: the battery system we chose to power the assembly was only working sometimes, and the waterproofing was rather sketchy. Firstly, we decided to use a 9-volt battery to power the assembly due to size and weight constraints. This only worked some of the time, and proved far less reliable than powering the arduino from a USB battery. Secondly, the water-resistant box we chose to put the components in was not as waterproof as we would've liked, and leaked quite easily. We would recommend investing in more robust waterproofing.

When putting together the sensing assembly and the boat, we ran into a few more problems. When we tried driving the boat with the waterproof box out front, we lost all steering capability, as the box acted as an external rudder. We fixed this by elevating the winch system so that the box could be pulled completely out of the water when the boat was moving around. We also needed to adjust for the new center of mass, as the sensing assembly was quite heavy, and pulled the boat forward a lot. So, we decided to put a piece of metal at the back of the boat to offset the center of mass, which also made the boat sit a lot lower in the water. We ultimately got the assembly and the boat to work together, but the boat lost a lot of its agility and speed in the process.



The final component of our project was the MATLAB code to visualize the depth of the body of water graphically rather than in a data table. The code simply would read values from a comma separated value (CSV) file provided by the depth sensor and graph the GPS location on the x and y axis and the depth of the body of water on the z axis. We wrote the code over a few hours in one night, and we definitely could have taken more time to make the data look smoother and work on the coloring of the graphs. We never got a chance to test the code with real data because of the leak in our diving component, but in the future this code could hopefully be used and tweaked to produce an accurate representation of the bottom of bodies of water.

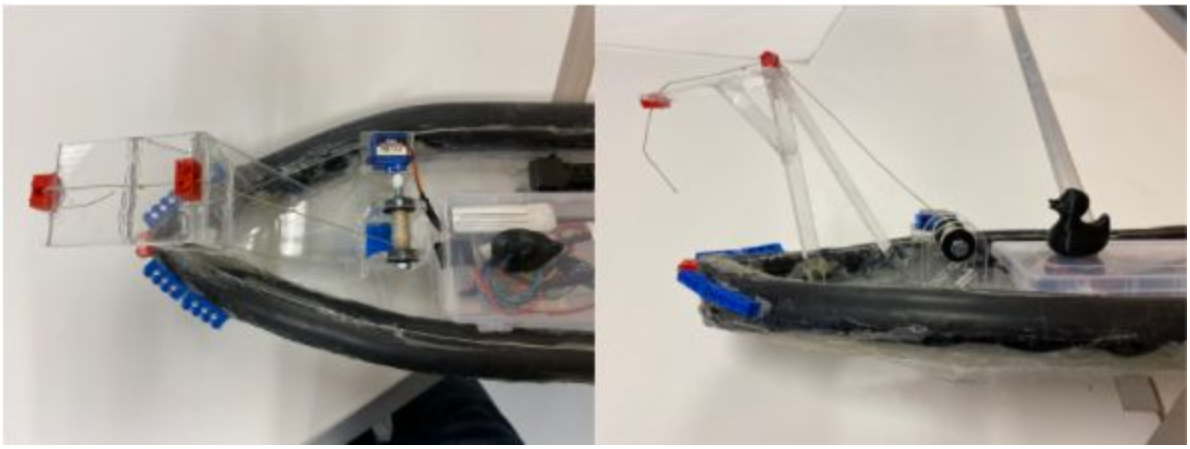




V. Description of Final Design

Our final design of the DSB was very similar to our original design and prototype with a few adjustments. The hull of the boat was made of acrylic, and it was designed in Onshape and laser cut through CorelDRAW². We used a bike tube wrapped around the boat as reinforcement for the boat's ability to float. Hot glue and a few rubber bands were used to attach the bike tire to the boat. We also had attached lego blocks to the front of the boat on the bike tire to help break up the water and prevent water from getting on the boat from the front. The final design also featured a counter weight on the back to help recenter the center of mass of the boat and prevent the boat from sinking.

The final design included a winch system to raise and lower our container of sensors. A mounted continuous servo was the way our winch system was powered and operated. Two pieces of acrylic were cut to host a set of 3-D printed ball bearings that ran a metal rod through them. The metal rod was then connected to the continuous servo with marine weld. String was wound up on the rod and connected to the container through two lego blocks to lessen the tension of the string on the ramp. The ramp was built with four pieces of laser cut acrylic to help hoist the container above the water to prevent it from acting as an additional rudder.



The container we used to lower our sensors contained a gps module, sd card reader, pressure sensor, voltage converter, 9v battery, jumper wires, Arduino nano, and clay. The container we bought was designated as a waterproof case for a phone so we had to cut a hole to be able to insert and hold our pressure sensor. A rubber washer was used to help seal off the hole between the pressure sensor and the container.

Another container was glued on top of the boat to host a breadboard, Arduino nano, bluetooth module, motor battery, and jumper wires. This container is not waterproof, but rather water resistant to help prevent the electronics from getting wet and to help keep them organized. This container has the electronics that control the motor and the rudder.

The final drivetrain of the boat consisted of a brushless motor for the propulsion and a servo to steer. After a couple prototypes for the propulsion system we settled on a brushless motor mounted to the rear of the boat above the water with a remote control plane propellor on the motor. The propeller had a four inch diameter and the combination of motor and propeller produced plenty of thrust to move the loaded down boat in the water. To mount the motor we cut acrylic into a platform to keep the motor elevated and 3D print a mount for the motor that was open for the motor to keep cool but also allowed us to securely attach it to the boat with zip-ties. For the steering we attached a servo to the deck of the boat that was connected to a lego drivetrain. The legos formed a driveshaft lookalike that went to a box that transferred horizontal motion to vertical. A rudder was at the end of the steering shaft and it had a full 180 degrees to move back and forth to steer the boat which proved to be more than we needed.

VI. Testing

The critical components of our project were the bluetooth control system for the boat and the submersible component for the depth measuring and data collection. Each of the components were tested individually and put together on the final project. Initially we had to create a working boat before even starting on the submersible components as a proof of concept. To test the boat's remote control capabilities we created a circuit for the motor and rudder on a small arduino. Individually the motor and rudder were easy to get working but the most testing was done when they were combined and controlled via bluetooth. We took multiple class periods to get the code right that allowed us to control both the motor and rudder reliably. Testing was mostly completed in the classroom, until we had the controls to a point where we trusted the boat to work on the water. Two days we tested on the water and the first brought out many key issues. The motor would stutter instead of running continuously, and the rudder had too much freedom to move back and forth so the boat was almost impossible to steer. Motivated by the fact that the boat floated and partially worked, we fixed the code for the motor so it would run continuously and predictably and then we started on the rudder. After a little research on how far a rudder needed to turn to steer a boat, we limited its movement to 30 degrees each way, and we made the rudder return to center after letting off the key to steer the boat with more control. The first test in the water presented key issues with the code, but not the hardware so they were quickly fixed and the boat was ready to test again the next day. Testing both of these went well in the water and we could move on to testing our data collection.

After testing and successfully creating a remote controllable boat, we had to test the submersible data collection component of the boat. The waterproof container it would go in was tested in a seven foot pool to check for leaks and it passed with flying colors. The GPS sensor was also tested outside the ITLL and it recorded location after a bit of testing with the code and transcribing the data. All we needed to test then was the depth sensor in a body of water. This is where we ran into problems, as we sank the housing for the sensor it began to leak and we had to get it out of the water before it destroyed our electronics and sensor. A lack of testing the waterproof container at a depth of more than seven feet prevented us from successfully completing our project. Without this proper testing the leaking issue was not brought to light till it was too late and our project did not function as planned. Testing overall occurred very frequently as many parts of the boat were not finalized until we tested them, but a lapse of judgement in testing the waterproof component in depth swiftly ended our data collection.

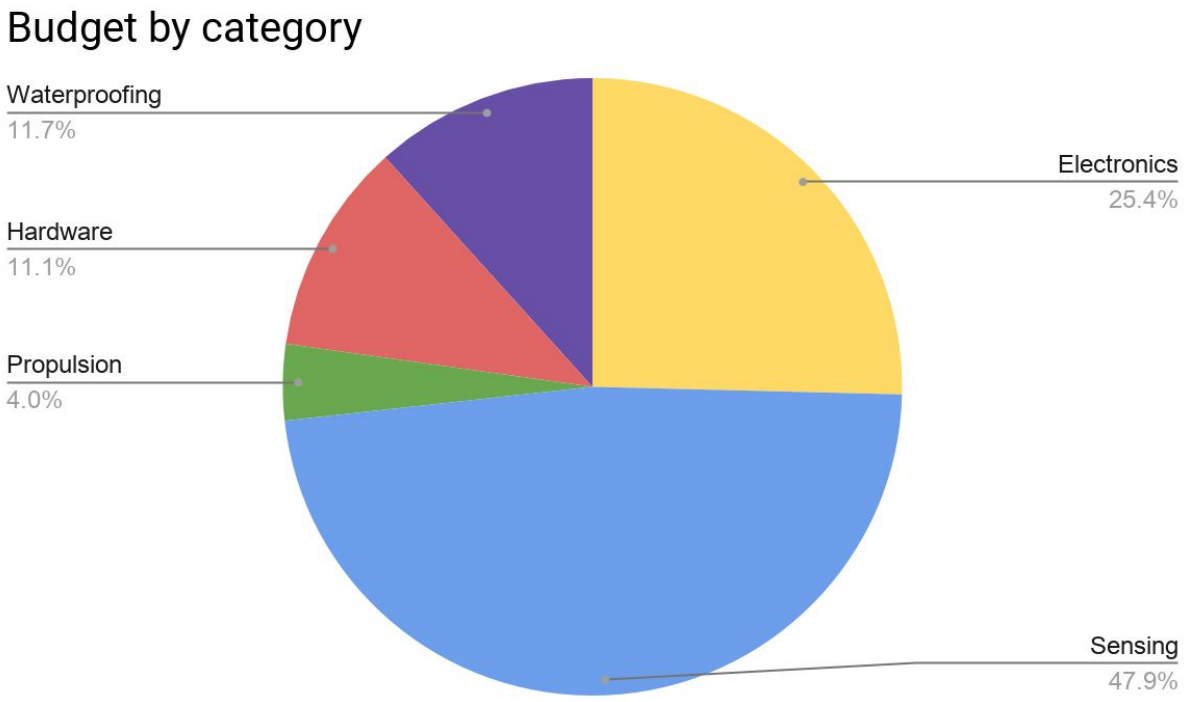
VII. Conclusions

We did a lot with this project in a relatively short amount of time. We were able to design and build an impressive-looking boat out of acrylic and a bike tire, and add a propulsion system and steering to it. We implemented bluetooth control using an Arduino, and later managed to drive the boat around a lake remotely, using a laptop. Additionally, we recycled a servo from a previous project, and utilized 3d-printed ball bearings to make a bluetooth-controlled winch at the front of the boat. Finally, we designed a sensing system that we submerged under the water to collect data, and we wrote software that processed and mapped that data. We completed all of the components of our design; the only trouble we had was putting them together. We had difficulty driving the boat with the sensing system attached to the front, as it pulled the boat's center of mass to the front. We also should've invested more into waterproofing the sensing assembly, as it leaked on one trial and very nearly ruined the electronics inside. If we had another week or two to finish the project before the end of the semester, we likely would have completed it in its entirety. However, we were cut about a week or two short due to COVID-19.

Our group is very happy with our final product, even though it certainly isn't everything we hoped it would be. We failed to map the bottom of Kittredge Pond, but we grew as engineers and as a team, and built a bluetooth-controlled boat together.

VIII. Budget and Bill of Materials

Item	Amount ($)	Category	Provided By
GPS module	11.99	Sensing	Ryan Slocum
Arduino Nano (x2)	8.99	Electronics	Ryan Slocum
Micro SD writer	7.99	Electronics	Ryan Slocum
Bluetooth Modules	12.59	Electronics	Ryan Slocum
3D Printing Filament	0.50	Hardware	Ryan Slocum
Pressure Sensor	72.00	Sensing	Medhaj Shrestha
Clay	4.00	Hardware	Medhaj Shrestha
Voltage Converter	15.00	Electronics	Medhaj Shrestha
Brushless Motor	0.00	Propulsion	Dr. Knutsen
Motor speed controller	0.00	Electronics	Dr. Knutsen
Motor Battery	0.00	Electronics	Dr. Knutsen
Acrylic Panels	15.00	Hardware	Jake Spagnolli
JB Weld Marine Adhesive	7.99	Waterproofing	Jake Spagnolli
Gorilla Glue Silicon	4.99	Waterproofing	Jake Spagnolli
Fishing Box	3.99	Waterproofing	Jake Spagnolli
Propellor	6.99	Propulsion	Connor Ford
Submersible box	3.50	Waterproofing	Connor Ford
Lego building blocks	0.00	Hardware	ITLL
Hot Glue	0.00	Hardware	ITLL
String	0.00	Hardware	ITLL
Heat Shrink	0.00	Hardware	ITLL
Steel Blocks	0.00	Hardware	ITLL
Total:	**175.52**		


Budget by category
Waterproofing
11.7%
Electronics
25.4%
Hardware
11.1%
Propulsion
4.0%
Sensing
47.9%

IX. Timeline

October				
5 Project Assigned	6 Initial Design Finalized	7	8 Brushless Motor Testing	9
12 Hull Prototype	13 Hull Final Build	14	15 Tire Tube Install Goal 1 Fulfilled	16
19	20 Rudder System Built	21	22 Preliminary Design Review	23
26	27 Propulsion System Finalized	28 Bluetooth Connection Done	29 Propulsion System Mounted	30

November				
2	3 First test on the water	4 Code & Bluetooth For Control Fixed	5 Second test on the water	6
9	10 Finished Winch First Drop Test	11	12 Fixed Winch Final Edits	13 MATLAB code worked on
16	17 Final testing Second Drop Test	18 Expo prep	19 Last day in-person Final Expo prep	20 Expo Video due
23 Expo Interview	24	25	26	27

X. Appendix

1. Processing IDE Bluetooth Broadcaster Code

```
import processing.serial.*;
Serial myPort;

void setup(){
  size(1000, 1000);
  myPort = new Serial(this, "COM4", 9600);
  myPort.bufferUntil('\n');
}
int rudderVal = 0;
int motorVal = 0;
int winchVal = 0;
void draw() {
 background(0, 90, 0);
 if (keyPressed) {
   if (key == 'a' || key == 'A') {
     rudderVal = 0;
   }
   if (key == 'd' || key == 'D') {
     rudderVal = 2;
   }
   if (key == 'q' || key == 'Q') {
     winchVal = 0;
   }
   if (key == 'e' || key == 'E') {
      winchVal = 2;
   }
   if (key == 'w' || key == 'W') {
     motorVal = 2;
   }
 } else {
    winchVal = 1;
    motorVal = 1;
    rudderVal = 1;
  }
 String data = String.valueOf(motorVal) + "," + String.valueOf(rudderVal) + "," + String.valueOf(winchVal) + "\n";
 println(data);
 myPort.write(motorVal*100 + rudderVal*10 + winchVal);
}
```

Above is the code for the Processing IDE. This code took in keyboard inputs, created an encoded value, and sent it over bluetooth Serial to the Arduino.

2. Arduino Bluetooth Receiver Code

```
#include <Servo.h>
Servo rudder;
Servo motor;
Servo winch;

void setup() {
  rudder.attach(5);
  motor.attach(6);
  winch.attach(7);

  Serial.begin(9600);
  motor.write(1000);
  delay(1000);
}
int state = 0;
int rudderVal = 0;
int winchVal = 91;
int motorVal = 1000;

void loop() {
  if(Serial.available() > 0){
    state = Serial.read();
  }
  if (state == 0) {
    state = 111;
  }
  winchVal = state % 10;
  rudderVal = (state % 100)/10;
  motorVal = (state / 100) - 1;
  winchVal = map(winchVal, 0, 2, 0, 180);
  rudderVal = map(rudderVal, 0, 2, 40, 110);
  motorVal = map(motorVal, 0, 1, 1000, 2000);
  Serial.println(state);
  winch.write(winchVal);
  rudder.write(rudderVal);
  motor.write(motorVal);
}
```

Above is the code for the Arduino. This code took in the values from the laptop, decoded them, and turned them into outputs on the winch, motor, and rudder.

3. MATLAB Graphing Code

```
clear all
clear
clc

A = [40.2 40.3 40.4 40.5 40.6 40.7; 110.2 110.3 110.4 110.5 110.6 110.7];
[X, Y] = meshgrid(A(1,:),A(2,:));

% this section is repalced by a read in of a csv file for the depth calc
Z = [1 3 7 12 9 4; 2 9 12 19 13 2; 1 4 9 17 9 3; 1 3 5 12 6 4; 2 5 8 10 12 5; 3 7 8 10 7 2];

Z = -Z;

C = Z;

surf(X,Y,Z,C)
colorbar
xlabel('Latitiude')
ylabel('Longitude')
zlabel('Depth (meters)')

hold on
figure
contour3(X,Y,Z)
colorbar
xlabel('Latitiude')
ylabel('Longitude')
zlabel('Depth (meters)')
```

This is the MATLAB plotting code for the test plots shown above, the real depth data would need to be brought in from a csv file from the SD card reader in the submersible component and would replace the Z matrix in the code.

4. Arduino GPS Code

PressureSensor_Gps-SD

```
#include <SPI.h>
#include <SD.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
#include <Wire.h>
#include "MS5837.h"
//file to write to
File myFile;
//pressure sensor object
MS5837 sensor;
static const int RXPin = 6, TXPin = 5;
static const uint32_t GPSBaud = 9600;
//software serial for recieving and transmitting gps data
SoftwareSerial ss(RXPin, TXPin);
//gps object
TinyGPSPlus gps;
void setup()
{
  ss.begin(GPSBaud);
  Serial.begin(9600);
  Wire.begin();
    while (!sensor.init())
    {
    Serial.println("Init failed!");
    Serial.println("Are SDA/SCL connected correctly?");
    Serial.println("Blue Robotics Bar30: White=SDA, Green=SCL");
    Serial.println("\n\n\n");
    delay(5000);
    }
  sensor.setModel(MS5837::MS5837_30BA);
  sensor.setFluidDensity(997);
```

```
  sensor.setFluidDensity(997);
  if (!SD.begin(10))
  {
  while (1);
  }
  if (SD.exists("test.csv")) {
    SD.remove("test.csv");
  }
  myFile = SD.open("test.csv", FILE_WRITE);
  if (myFile) {
  myFile.println("Latitude,Longitude, Speed, hour, min, sec, Pressure, Temperature, Depth");
  //Serial.println("Millis,Potentiometer Value");
  }
}
void loop()
{
  if (millis() < 10000)
  {
 sensor.read();
 smartDelay(1000);
 myFile.print(gps.location.lat(), 6);
 myFile.print(",");
 Serial.print(gps.location.lat(),6);
 Serial.print(",");
 myFile.print(gps.location.lng(), 6);
 myFile.print(",");
 Serial.print(gps.location.lng(),6);
 Serial.print(",");
 myFile.print(gps.speed.mph());
```

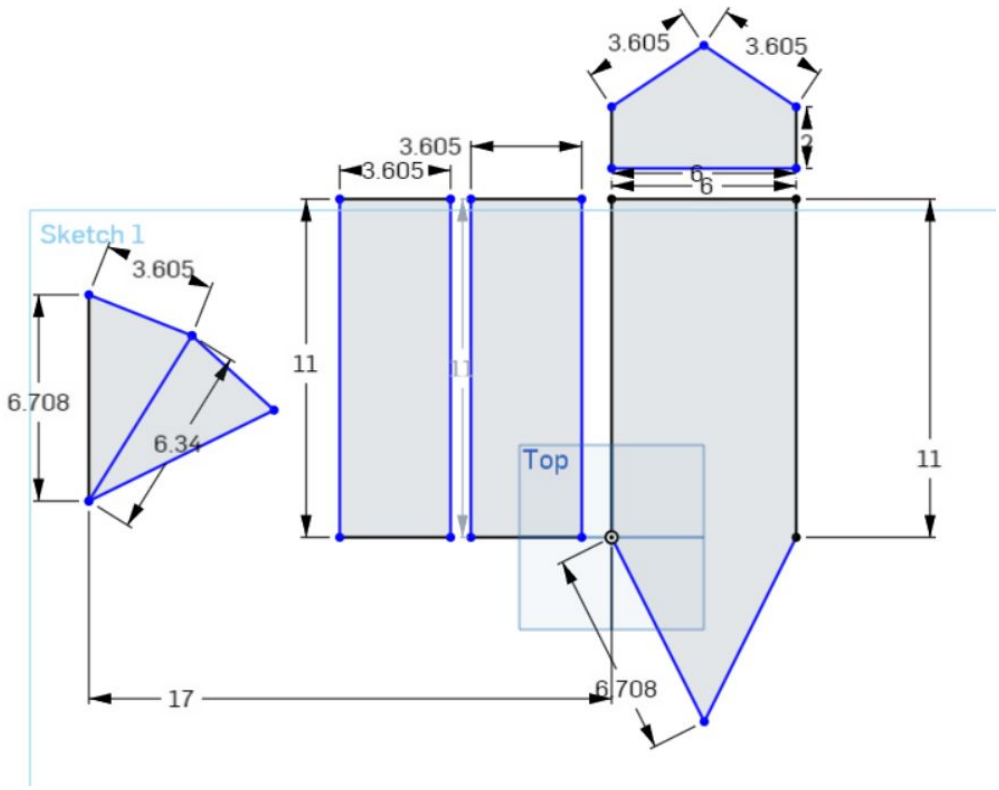
```
 myFile.print(gps.speed.mph());
 myFile.print(",");
 Serial.print(gps.altitude.feet(),6);
 Serial.print(",");
 myFile.print(gps.time.hour());
 myFile.print(",");
 myFile.print(gps.time.minute());
 myFile.print(",");
 myFile.print(gps.time.second());
 myFile.print(",");
 Serial.print("Pressure Sensor: ");
 myFile.print(sensor.pressure()); //mbar
 myFile.print(",");
 Serial.print(sensor.pressure());
 Serial.print(",");
 myFile.print(sensor.temperature()); //C
 myFile.print(",");
 Serial.print(sensor.temperature());
 Serial.print(",");
 myFile.println(sensor.depth()); //m
 Serial.println(sensor.depth());

  }
  else
  {
    myFile.close();
  }
}
static void smartDelay(unsigned long ms)
{
  unsigned long start = millis();
```

```
{
  unsigned long start = millis();
  do
  {
    while (ss.available())
      gps.encode(ss.read());
  } while (millis() - start < ms);
}
```

The arduino code above is utilized to retrieve and parse specific GPS data, retrieve pressure sensor data (pressure, depth, temperature), and write the data to an SD card. (Serial.print was used to troubleshoot).

5. CAD files



Onshape file with dimensions for the pieces of the hull.